# RegretCache, Explained Simply

## 1. The idea in 20 seconds

A cache is a small shelf. When it is full, something must be thrown off to make room. The usual rule (LRU) throws off whatever was used longest ago.

**RegretCache = LRU + a memory of its own mistakes.** It remembers the last few things it threw off. If one of them is asked for again soon, that was a mistake, and that item gets a **protection score** so it is harder to throw off next time.

## 2. The parts

| Part | What it is |
|---|---|
| **Cache** | The shelf. Holds `capacity` pages. |
| **Ghost list** | A short list (`ghost_size`) of recently evicted pages. Only their names are kept, not the data. |
| **Score** | A number per page. Higher means more protected. Starts at 0. |
| **Decay** | Scores fade: they halve every `half_life` requests (default 100). |
| **Alpha** | How big one lesson is. It changes with how many recent mistakes there were. |

## 3. What happens on every request

1. **Hit** (page is in the cache): note that it was just used. Done.
2. **Ghost hit** (page is not in the cache, but is on the ghost list): this is a **mistake**. Remove it from the ghost list and raise its score by `alpha × (1 / gap)`. The gap is how many requests ago it was evicted.
3. **Plain miss** (page is nowhere): nothing to learn.
4. **If the cache is full, evict** the page with the **lowest score**. If scores tie, evict the one used longest ago. With all scores at 0, this is exactly LRU.
5. **Remember the eviction:** put the evicted page on the ghost list. If the list is too long, drop its oldest entry and forget that page's score.
6. **Insert** the requested page.

Note the order: the eviction happens before the new page goes in, so a page that just came back from the ghost list can't be thrown out in the same step.

## 4. The two formulas

**Size of a lesson:** `score += alpha × (1 / gap)` Back after 1 request: +alpha. Back after 20: +alpha/20. A quick return is a bigger mistake, so it gets more protection.

**Alpha (how much to learn):**

- Look at the last 10 evictions and count how many turned out to be mistakes. `rate = mistakes / 10`.
- For the first 10 evictions, alpha = 0.5.
- After that, `alpha = 0.25 + 0.75 × rate`, so it runs from 0.25 (no recent mistakes) to 1.0 (all mistakes).
- In short: when it is doing badly, it learns faster.

**Decay:** `score_now = score × 0.5^(age / half_life)`. Old lessons fade but never reach exactly 0. Scores are also capped between -1 and 5, though in practice they never go near either limit.

## 5. A tiny example

Cache size 3, ghost size 3, requests `1 2 3 4 1 5 6 7 1`.

- t=4: page 4 arrives, cache is full, page 1 is evicted (oldest). Ghost list: 1.
- t=5: page 1 is requested. It is on the ghost list, so that was a mistake. The gap is 1 and alpha is 0.5, so its score becomes 0.5. Page 2 is evicted instead.
- t=6, 7: pages 3, then 4, are evicted (score 0, oldest first).
- t=8: LRU would evict page 1 now, because it is the oldest. RegretCache sees page 1 has a score of about 0.49 and evicts page 5 instead.
- t=9: page 1 is requested and it is a **hit**.

Result: FIFO 0 hits, LRU 0, **RegretCache 1**, Optimal 2.

## 6. When it works and when it doesn't

- **Works well:** a few popular pages mixed with a stream of one-time pages (scan + hot set). LRU keeps throwing the popular pages out. RegretCache sees them come back, protects them, and evicts the junk. It beat LRU in 30 of 30 test runs.
- **Works somewhat:** zipf (skewed popularity), 28 of 30 runs.
- **Mixed:** shifting popularity, 15 wins and 13 losses. Don't claim it reliably wins here.
- **No help:** a loop bigger than the cache. It gives exactly LRU's result.

## 7. What is new about it?

The novelty is measured against the standard baselines: **FIFO, LRU and Optimal**.

**What it does differently from LRU and FIFO:**

| Algorithm | How it picks what to throw off |
|---|---|
| FIFO | The page that arrived first. It ignores use entirely. |
| LRU | The page used longest ago. It never checks whether that choice was right. |
| **RegretCache** | **The page with the lowest protection score, with LRU as the tie-break. The score comes from its own past mistakes.** |

So the novelty is the **per-page, regret-weighted protection score**:

1. **It checks its own evictions.** LRU and FIFO never find out they threw off something needed. RegretCache keeps a ghost list so it can find out.
2. **The lesson sticks to the exact page that was wronged.** That page gets protected, not a whole policy or a whole list.
3. **The lesson is weighted by how fast the page came back (`1/gap`).** A quick return is a bigger mistake.
4. **Lessons fade over time**, so it can let go when popularity changes.
5. **With no mistakes recorded, it is exactly LRU.** It only departs from LRU where it has evidence.

**What the results show against the standard baselines:**

- It beat LRU in 30 of 30 runs on scan + hot set and 28 of 30 on zipf, and it matched LRU exactly on loop.
- Shifting is mixed (15 wins, 13 losses), so don't claim it wins there.
- "Gap closed" shows how much of the distance from LRU to Optimal it covered: about 69% on scan + hot, 33% on zipf, 15% on shifting, and 0% on loop.

**How to say it safely:** "RegretCache is LRU plus a memory of its own eviction mistakes. When an evicted page comes back soon, that page gets a fading protection score, bigger the sooner it returned. I measured it against FIFO, LRU and Optimal on four synthetic workloads: it clearly beats LRU on scan + hot set and zipf, is mixed on shifting, and ties on loop."

Don't say "it beats everything" or "it works on real traffic." It was only tested on made-up workloads.

## 8. Questions you may get

**Why does 1/gap matter?** A page that comes straight back was clearly needed. Replacing it with a flat bonus made zipf results worse.

**Why do scores fade?** So old favourites don't block new ones. Without fading, shifting popularity goes from +15% to -12% of the gap to Optimal closed (worse than LRU).

**Does the half-life slider do anything?** Almost nothing. A faded score never reaches 0, so a once-protected page always outranks an unprotected one. Turning decay off does matter.

**Does the adaptive alpha help?** Not measurably. A fixed alpha of 0.5 gives the same hit rates within 0.2 points.

**Where do 0.25, 0.75, 0.5 and 10 come from?** They are design guesses, not derived or tuned. Results barely change if you change them.

**Why compare with Optimal?** It sees the future, so it is the best any cache can do. "Gap closed" tells you how much of the distance from LRU to Optimal you covered.

**Is it practical?** Not as written. Each eviction sorts the whole cache, and it is about 7x slower than LRU per request. Fine for a simulator, bad for thousands of slots.

**What are the limits of the evidence?** Only synthetic workloads, 300 requests per run, 10 seeds, cache sizes 4, 8 and 16.

## 9. One-line summary

**RegretCache evicts like LRU, but it remembers evictions it regrets, and gives those pages a fading protection score that is bigger the sooner they came back.**